

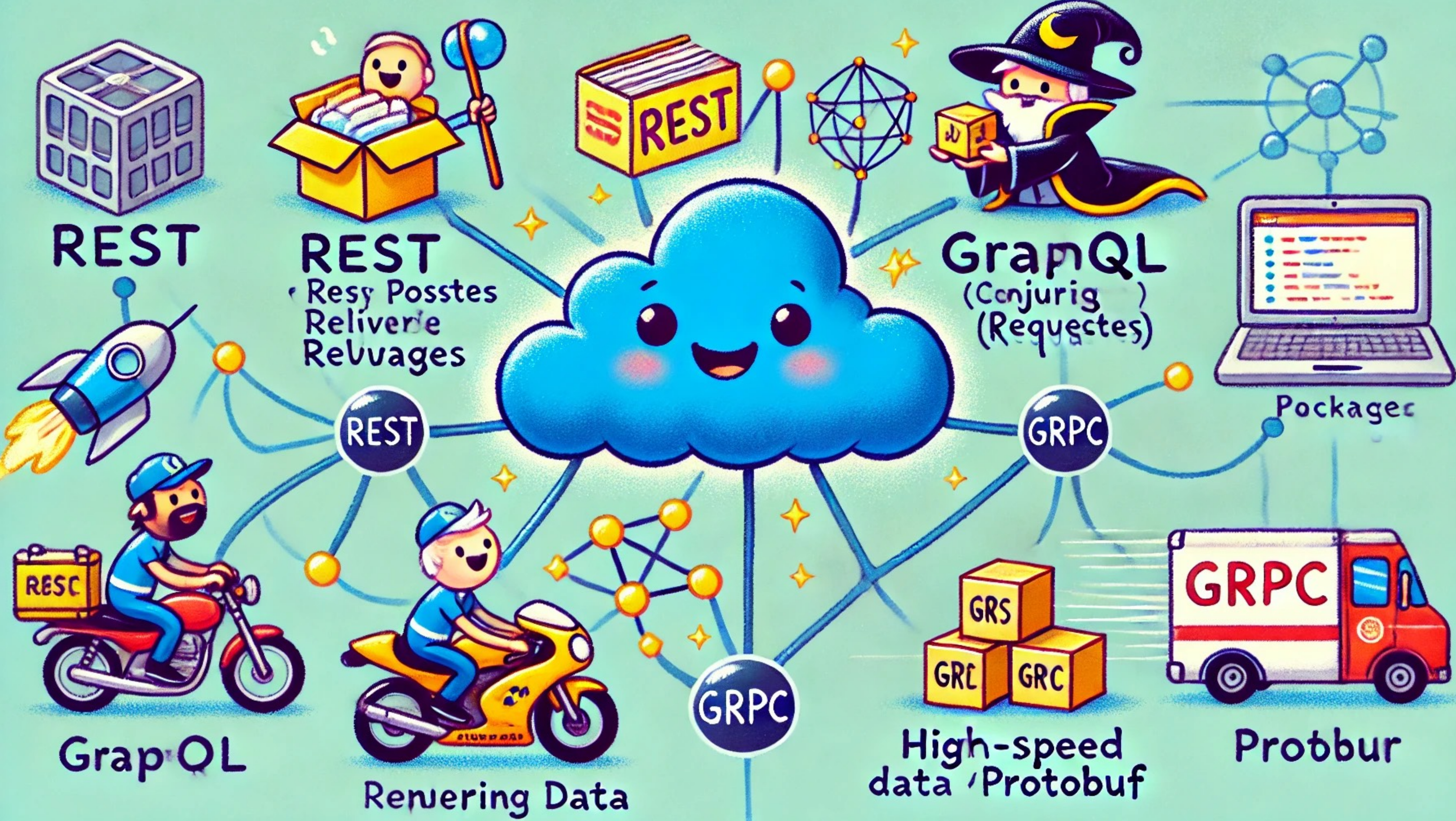
Cloud Computing Technologies

DAT515 - Fall 2026

Cloud APIs

Prof. Hein Meling





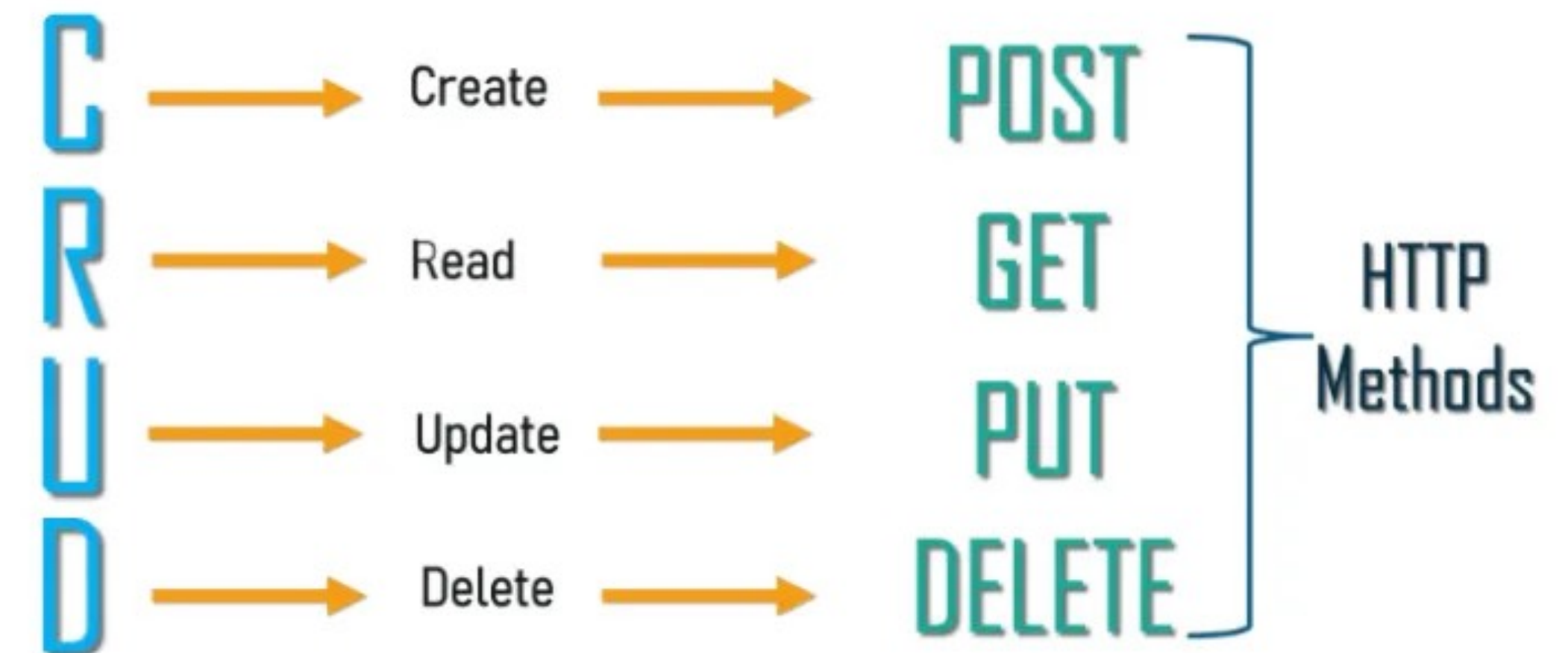
Cloud APIs

- REST
- GraphQL
- gRPC

REST APIs

REST APIs

- REST is a web standards-based architecture that uses HTTP
- **Stateless**: Each request from client to server must contain all necessary information
- **CRUD** operations
- Operates on resources identified by URIs
- Widely supported and simple to use



Routing Requests

Most Basic Web Server (net/http)

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "Hello, you've requested: %s\n", r.URL.Path)
    })

    http.ListenAndServe(":80", nil)
}
```

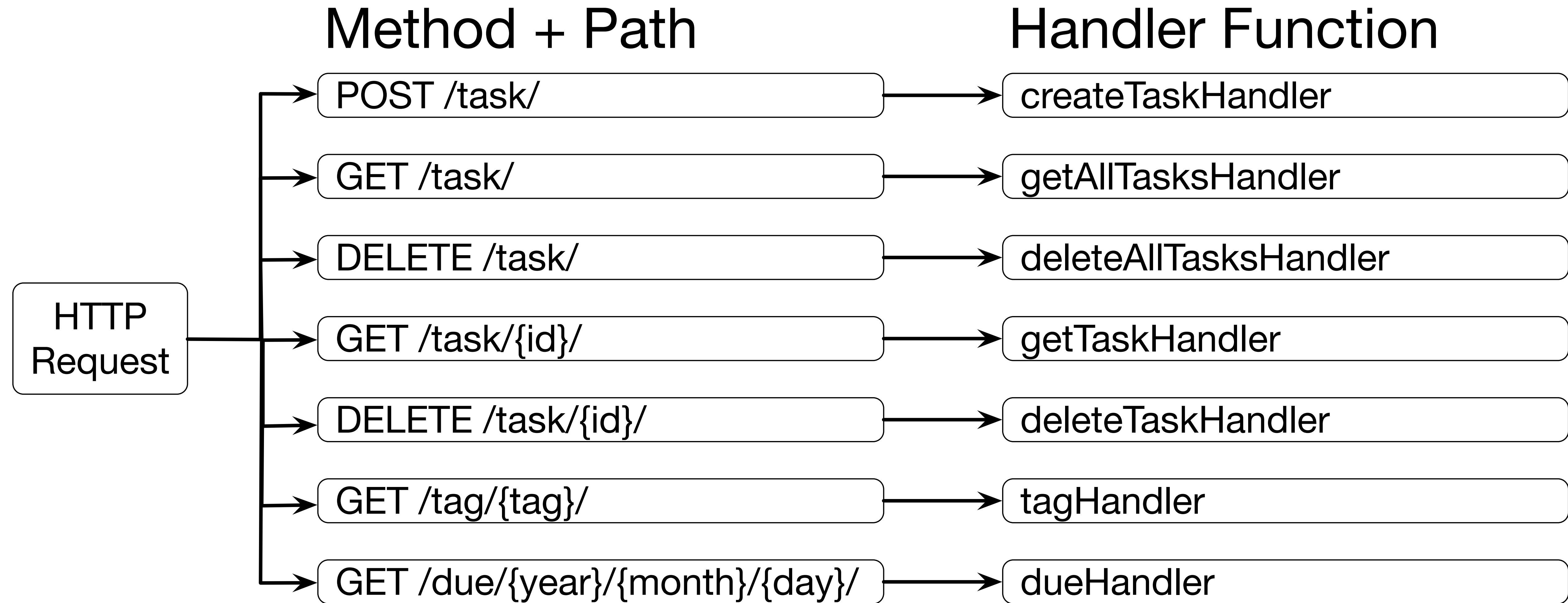
```

mux := http.NewServeMux()
server := NewTaskServer()

mux.HandleFunc("POST /task/", server.createTaskHandler)
mux.HandleFunc("GET /task/", server.getAllTasksHandler)
mux.HandleFunc("DELETE /task/", server.deleteAllTasksHandler)
mux.HandleFunc("GET /task/{id}/", server.getTaskHandler)
mux.HandleFunc("DELETE /task/{id}/", server.deleteTaskHandler)
mux.HandleFunc("GET /tag/{tag}/", server.tagHandler)
mux.HandleFunc("GET /due/{year}/{month}/{day}/", server.dueHandler)

```

Example based on Eli Bendersky's [blog post](#) (see link for full code)



Web Server with Stdlib (net/http)

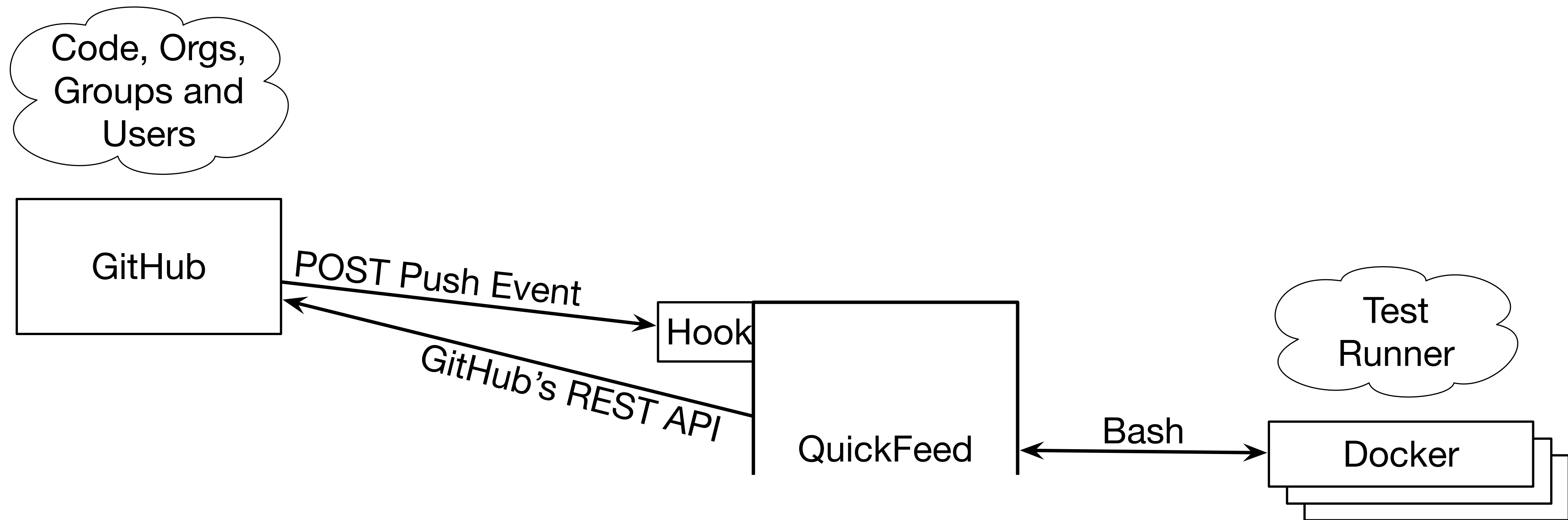
```
func (ts *taskServer) getTaskHandler(w http.ResponseWriter, req *http.Request) {
    log.Printf("handling get task at %s\n", req.URL.Path)

    id, err := strconv.Atoi(req.PathValue("id"))
    if err != nil {
        http.Error(w, "invalid id", http.StatusBadRequest)
        return
    }

    task, err := ts.store.GetTask(id)
    if err != nil {
        http.Error(w, err.Error(), http.StatusNotFound)
        return
    }

    renderJSON(w, task)
}
```

QuickFeed as an Example



```
// RegisterRouter registers http endpoints for authentication API and scm provider webhooks.
```

```
func (s *QuickFeedService) RegisterRouter(tm *auth.TokenManager, authConfig *oauth2.Config, public string) *http.ServeMux {
```

```
    // Serve static files.
```

```
    router := http.NewServeMux()
```

```
    assets := http.FileServer(http.Dir(public + "/assets")) // skipcq: GO-S1034
```

```
    dist := http.FileServer(http.Dir(public + "/dist"))      // skipcq: GO-S1034
```

```
    router.Handle("/", http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
```

```
        | http.ServeFile(w, r, public+"/assets/index.html")
```

```
    })))
```

```
    paths, handler := s.NewQuickFeedHandler(tm)
```

```
    router.Handle(paths, controller(handler, streamTimeout))
```

```
    router.Handle(auth.Assets, http.StripPrefix(auth.Assets, assets))
```

```
    router.Handle(auth.Static, http.StripPrefix(auth.Static, dist))
```

```
    // Register auth endpoints.
```

```
    callbackSecret := rand.String()
```

```
    router.HandleFunc(auth.Auth, auth.OAuth2Login(s.logger, authConfig, callbackSecret))
```

```
    router.HandleFunc(auth.Callback, auth.OAuth2Callback(s.logger, s.db, tm, authConfig, callbackSecret))
```

```
    router.HandleFunc(auth.Logout, auth.OAuth2Logout())
```

```
    // Register hooks.
```

```
    ghHook := hooks.NewGitHubWebHook(s.logger, s.db, s.scmMgr, s.runner, s.bh.Secret, s.streams, tm)
```

```
    router.HandleFunc(auth.Hook, ghHook.Handle())
```

```
    return router
```

```
}
```

```
const (
```

```
    Cookie           = "cookie"
```

```
    CookieName       = "auth"
```

```
    SetCookie        = "Set-Cookie"
```

```
    tokenExpirationTime = 15 * time.Mi
```

```
    cookieExpirationTime = 24 * time.Ho
```

```
    alg               = "HS256"
```

```
    githubUserAPI = "https://api.github
```

```
// Routes
```

```
Auth      = "/auth/"
```

```
Teacher   = "/auth/teacher/"
```

```
Callback  = "/auth/callback/"
```

```
Logout    = "/logout"
```

```
Hook      = "/hook/"
```

```
Assets    = "/assets/"
```

```
Static    = "/static/"
```

```
)
```

```
// Handle take POST requests from GitHub, representing Push events
// associated with course repositories, which then triggers various
// actions on the QuickFeed backend.
func (wh GitHubWebHook) Handle() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        payload, err := github.ValidatePayload(r, []byte(wh.secret))
        if err != nil {
            wh.logger.Error("invalid webhook request body", label.Error, err)
            w.WriteHeader(http.StatusUnauthorized)
            return
        }
        defer r.Body.Close()

        event, err := github.ParseWebHook(github.WebHookType(r), payload)
        if err != nil {
            wh.logger.Error("failed to parse GitHub webhook", label.Error, err)
            w.WriteHeader(http.StatusBadRequest)
            return
        }
        ctx := context.WithoutCancel(r.Context())
        ctx, logger := qlog.WithLogger(qlog.NewContext(ctx, wh.logger), eventTypeLabel, github.WebHookType(r))
        logger.Debug("received webhook event")

        switch e := event.(type) {
        case *github.PushEvent:
            commitID := e.GetHeadCommit().GetID()
            ctx, logger := qlog.WithLogger(ctx, label.Commit, commitID)
            logger.Debug("received push event")
        }
    }
}
```

```
import (
    "context"
    "log/slog"
    "net/http"
    "time"

    "github.com/google/go-github/v62/github"
    "github.com/quickfeed/quickfeed/ci"
    "github.com/quickfeed/quickfeed/database"
```

```
2024-08-23T09:20:30.741+0200    DEBUG    hooks/github.go:66    {
  "ref": "refs/heads/main",
  "commits": [
    {
      "message": "Siste gang forhåpentilgvis",
      "author": {
        "name": "Christopher Liebich Kolle",
        "email": "cl.kolle@stud.uis.no",
        "username": "CKolle"
      },
      "url": "https://github.com/dat240-2024/CKolle-labs/commit/b411f6719158d095cff00b1edfc5d331663e8c62",
      "distinct": true,
      "id": "b411f6719158d095cff00b1edfc5d331663e8c62",
      "tree_id": "76bd3ecbea490d4b86c50e8d4ddee41f863f6ad0",
      "timestamp": "2024-08-23T09:20:26+02:00",
      "committer": {
        "name": "Christopher Liebich Kolle",
        "email": "cl.kolle@stud.uis.no",
        "username": "CKolle"
      },
      "modified": [
        "Lab1/UiS.Dat240.Lab1/Program.cs",
        "Lab1/UiS.Dat240.Lab1/TestSubmissions.cs"
      ]
    }
  ],
  "before": "3c2112adbbf2c9368f3946602522cdec0abbc1f1",
  "after": "b411f6719158d095cff00b1edfc5d331663e8c62",
  "created": false,
  "deleted": false,
  "forced": false,
  "compare": "https://github.com/dat240-2024/CKolle-labs/compare/3c2112adbbf2...b411f6719158",
  "repository": {
```

GitHub's REST API

REST API

API Version: 2022-11-28 (latest) ▾

- Migrations ▾
- Organizations ▾
- Packages ▾
- Pages ▾
- Projects (classic) ▾
- Pull requests ▾
- Rate limit ▾
- Reactions ▾
- Releases ▾
- Repositories ▴
- Autolinks

The REST API is now versioned. For more information, see "[About API versioning](#)."

REST API /

REST API endpoints for repositories

Use the REST API to create, manage and control the workflow of public and private GitHub repositories.

[REST API endpoints for repositories](#)

[List organization repositories](#)

[Create an organization repository](#)

[Get a repository](#)

[Update a repository](#)

[Delete a repository](#)

<https://docs.github.com/en/rest/repos?apiVersion=2022-11-28>

REST API endpoints for repositories

Use the REST API to manage repositories on GitHub.

```
gh api /orgs/dat515-2026/repos
```

List organization repositories [↗](#)

Lists repositories for the specified organization.

[i](#) Note

In order to see the `security_and_analysis` block for a repository you must have admin permissions for the repository or be an owner or security manager for the organization that owns the repository. For more information, see "[Managing security managers in your organization](#)."

Fine-grained access tokens for "List organization repositories"

This endpoint works with the following fine-grained token types:

- [GitHub App user access tokens](#)
- [GitHub App installation access tokens](#)
- [Fine-grained personal access tokens](#)

The fine-grained token must have the following permission set:

- "Metadata" repository permissions (read)

Code samples for "List organization repositories"

Request example

GET /orgs/{org}/repos

cURL JavaScript **GitHub CLI**

```
# GitHub CLI api
# https://cli.github.com/manual/gh_api

gh api \
  -H "Accept: application/vnd.github+json" \
  -H "X-GitHub-API-Version: 2022-11-28" \
  /orgs/ORG/repos
```

Response

Example response

Response schema

Status: 200

```
[
  {
    "id": 841558548,
    "node_id": "R_kgDOMikqFA",
    "name": "info",
    "full_name": "dat515-2024/info",
    "private": false,
    "owner": {
      "login": "dat515-2024",
      "id": 178194774,
      "node_id": "O_kgD0Cp8JVg",
      "avatar_url": "https://avatars.githubusercontent.com/u/178194774?v=4",
      "gravatar_id": "",
      "url": "https://api.github.com/users/dat515-2024",
      "html_url": "https://github.com/dat515-2024",
      "followers_url": "https://api.github.com/users/dat515-2024/followers",
      "following_url": "https://api.github.com/users/dat515-2024/following{/other_user}",
      "gists_url": "https://api.github.com/users/dat515-2024/gists{/gist_id}",
      "starred_url": "https://api.github.com/users/dat515-2024/starred{/owner}/{repo}",
      "subscriptions_url": "https://api.github.com/users/dat515-2024/subscriptions",
      "organizations_url": "https://api.github.com/users/dat515-2024/orgs",
      "repos_url": "https://api.github.com/users/dat515-2024/repos",
      "events_url": "https://api.github.com/users/dat515-2024/events{/privacy}",
      "received_events_url": "https://api.github.com/users/dat515-2024/received_events",
      "type": "Organization",
      "site_admin": false
    },
    "html_url": "https://github.com/dat515-2024/info",
    "description": null,
    "fork": false,
    "source": null,
    "parent": null,
    "is_template": false,
    "topics": null,
    "visibility": "public",
    "created_at": "2024-05-15T12:12:12Z",
    "updated_at": "2024-05-15T12:12:12Z",
    "pushed_at": "2024-05-15T12:12:12Z",
    "archived": false,
    "disabled": false,
    "open_issues": 0,
    "closed_issues": 0,
    "watchers": 0,
    "current_default_branch": null
  }
]
```

 GitHub Apps

 OAuth Apps

 Personal access tokens 

Fine-grained tokens

Beta

Tokens (classic)

Personal access tokens (classic)

Generate new token ▼

Revoke all

Tokens you have generated that can be used to access the [GitHub API](#).

Cloud API lecture token — *repo, user*

Last used within the last week

Delete

Expires on Tue, Oct 15 2024.

Navigate to **Settings**

Developer Settings

Personal Access Tokens

Fine-grained tokens or Tokens (classic)

Generate new token

Save token string securely

OR:

gh auth login

```
// createForkedRepo creates a forked repository from the assignments repository.
func (s *GithubSCM) createForkedRepo(ctx context.Context, opt *CreateRepositoryOptions) (*Repository, error) {
    const op Op = "createForkedRepo"
    m := M("failed to create forked repository")
    if !opt.valid() {
        return nil, E(op, m, fmt.Errorf("missing fields: %+v", *opt))
    }

    logger := qlog.FromContext(ctx).With(label.Repository, opt.Repo, label.Owner, opt.Owner)
    repo, resp, err := s.client.Repositories.Get(ctx, opt.Owner, opt.Repo)
    if err == nil {
        logger.Debug("forked repository already exists", "url", repo.GetHTMLURL())
        return toRepository(repo), nil
    }
    if !hasStatus(resp, http.StatusNotFound) {
        return nil, E(op, m, err)
    }

    logger.Debug("forking repository", "source_repository", qf.AssignmentsRepo)
    _, resp, forkErr := s.client.Repositories.CreateFork(ctx, opt.Owner, qf.AssignmentsRepo, &github.RepositoryCreateForkOptions{
        Organization: opt.Owner,
        Name:          opt.Repo,
    })
    if forkErr != nil && !hasStatus(resp, http.StatusAccepted) {
        return nil, E(op, M("failed to create fork %s/%s", opt.Owner, opt.Repo), forkErr)
    }
}
```

```
// Get fetches a repository.
//
// GitHub API docs: https://docs.github.com/rest/repos/repos#get-a-repository
//
//meta:operation GET /repos/{owner}/{repo}
func (s *RepositoriesService) Get(ctx context.Context, owner, repo string) (*Repository, *Response, error) {
    u := fmt.Sprintf("repos/%v/%v", owner, repo)
    req, err := s.client.NewRequest("GET", u, nil)

    if err != nil {
        return nil, nil, err
    }

    // TODO: remove custom Accept header when the license support fully launches
    // https://docs.github.com/rest/licenses/#get-a-repositorys-license
    acceptHeaders := []string{
        mediaTypeCodesOfConductPreview,
        mediaTypeTopicsPreview,
        mediaTypeRepositoryTemplatePreview,
        mediaTypeRepositoryVisibilityPreview,
    }
    req.Header.Set("Accept", strings.Join(acceptHeaders, ", "))

    repository := new(Repository)
    resp, err := s.client.Do(ctx, req, repository)
    if err != nil {
        return nil, resp, err
    }

    return repository, resp, nil
}
```

go-github is a Go client library for accessing the GitHub API v3

<https://github.com/google/go-github>

To start using it, run:

\$ go get github.com/google/go-github/v90

In Go code:

```
import "github.com/google/go-github/v90/github"
```

```
// SCM is the source code management interface for managing courses and users.
type SCM interface {
    // GetUserByID fetches a user by their SCM user ID.
    GetUserByID(context.Context, uint64) (*qf.User, error)

    // Gets an organization.
    GetOrganization(context.Context, *OrganizationOptions) (*qf.Organization, error)
    // Get repositories within organization.
    GetRepositories(context.Context, string) ([]*Repository, error)
    // Returns the number of commits the repository is ahead of assignments.
    CommitsAhead(context.Context, *RepositoryOptions) (int, error)

    // CreateCourse creates repositories for a new course.
    CreateCourse(context.Context, *CourseOptions) ([]*Repository, error)
    // UpdateEnrollment updates organization membership and creates and grants access to user repository.
    UpdateEnrollment(context.Context, *UpdateEnrollmentOptions) (*Repository, error)
    // RejectEnrollment removes user's repository and revokes user's membership in the course organization.
    RejectEnrollment(context.Context, *RejectEnrollmentOptions) error
    // DemoteTeacherToStudent revokes a user's owner status in the organization.
    DemoteTeacherToStudent(context.Context, *UpdateEnrollmentOptions) error
    // CreateGroup creates repository for a new group.
    CreateGroup(context.Context, *GroupOptions) (*Repository, error)
    // UpdateGroupMembers adds or removes members of an existing group.
    UpdateGroupMembers(context.Context, *GroupOptions) error
    // DeleteGroup deletes group's repository.
    DeleteGroup(context.Context, uint64) error
    // SyncFork syncs a forked repository's branch with its upstream repository.
    SyncFork(context.Context, *SyncForkOptions) error
}
```

Joke time

I asked the cloud API for
some space.

It provisioned 2 TB and
started billing me monthly.

What's the most frustrating part
of cloud API development?

When the response says
'200 OK,' but your brain says
'404 Not Found.'

GraphQL

Query language for APIs and runtime to execute those queries

- Allows clients to request **only the data they** need
 - No over-fetching or under-fetching
- Single endpoint: `/graphql`
- Supports nested queries and mutations
- Developed by Facebook for their news feed
 - (now used widely across services)

Query

```

{
  hero {
    name
    friends {
      name
    }
  }
}

```

Response

```

{
  "data": {
    "hero": {
      "name": "R2-D2",
      "friends": [
        {"name": "Luke Skywalker"},
        {"name": "Han Solo"},
        {"name": "Leia Organa"}
      ]
    }
  }
}

```

- GitHub's API v4
 - <https://github.com/shurcool/githubv4>
- Twitter, Shopify, Netflix, Airbnb, PayPal

QuickFeed Uses GitHub's GraphQL API

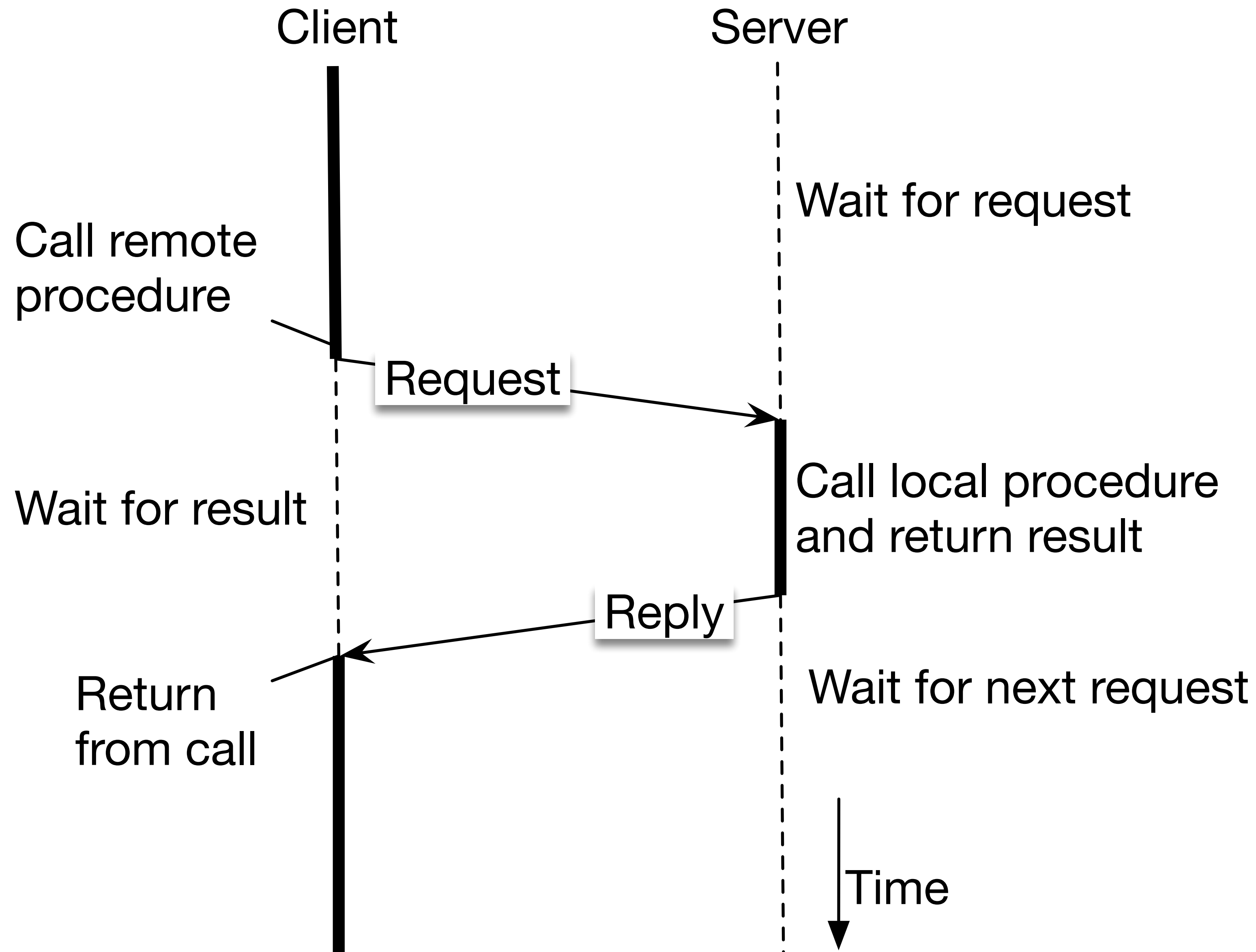
```
func (s *GithubSCM) DeleteIssue(ctx context.Context, opt *RepositoryOptions, issueNumber int) error {
    var q struct {
        Repository struct {
            Issue struct {
                ID githubv4.ID
            } `graphql:"issue(number:$issueNumber)"`
        } `graphql:"repository(owner:$repositoryOwner,name:$repositoryName)"`
    }
    variables := map[string]interface{}{
        "repositoryOwner": githubv4.String(opt.Owner),
        "repositoryName":  githubv4.String(opt.Repo),
        "issueNumber":      githubv4.Int(issueNumber),
    }
    if err := s.clientV4.Query(ctx, &q, variables); err != nil {
        return err
    }

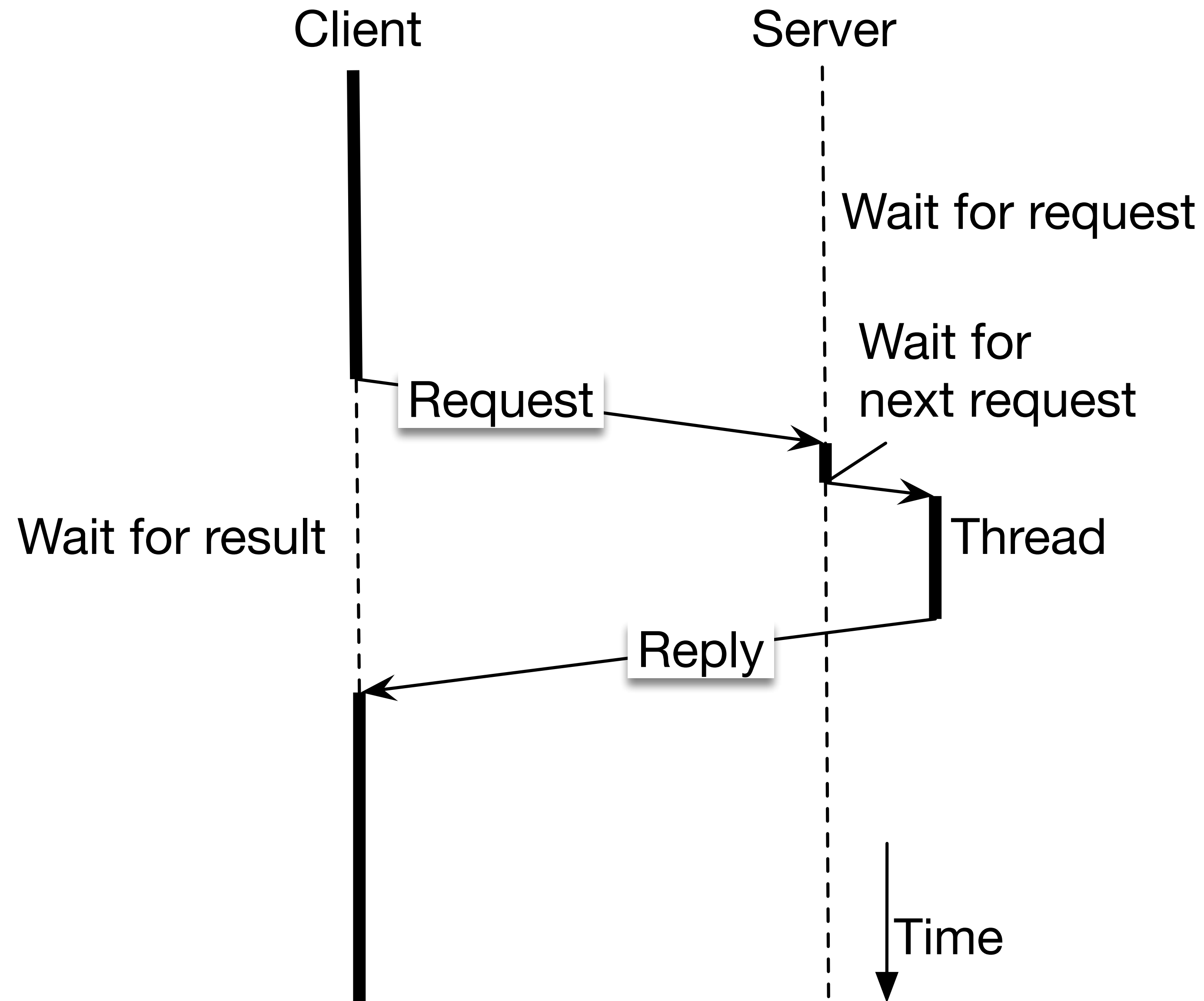
    var m struct {
        DeleteIssue struct {
            Repository struct {
                Name string
            }
        } `graphql:"deleteIssue(input:$input)"`
    }
    input := githubv4.DeleteIssueInput{
        IssueID: q.Repository.Issue.ID,
    }
    return s.clientV4.Mutate(ctx, &m, input, nil)
}
```

- Complex queries can result in **performance bottlenecks**
 - Servers must fetch and assemble from multiple data sources
 - (server-side over-fetching)
- Caching Challenges
 - Caching each REST resource is easy — has a unique URL (GET `/users/1`)
 - In GraphQL, traditional caching mechanisms become less effective

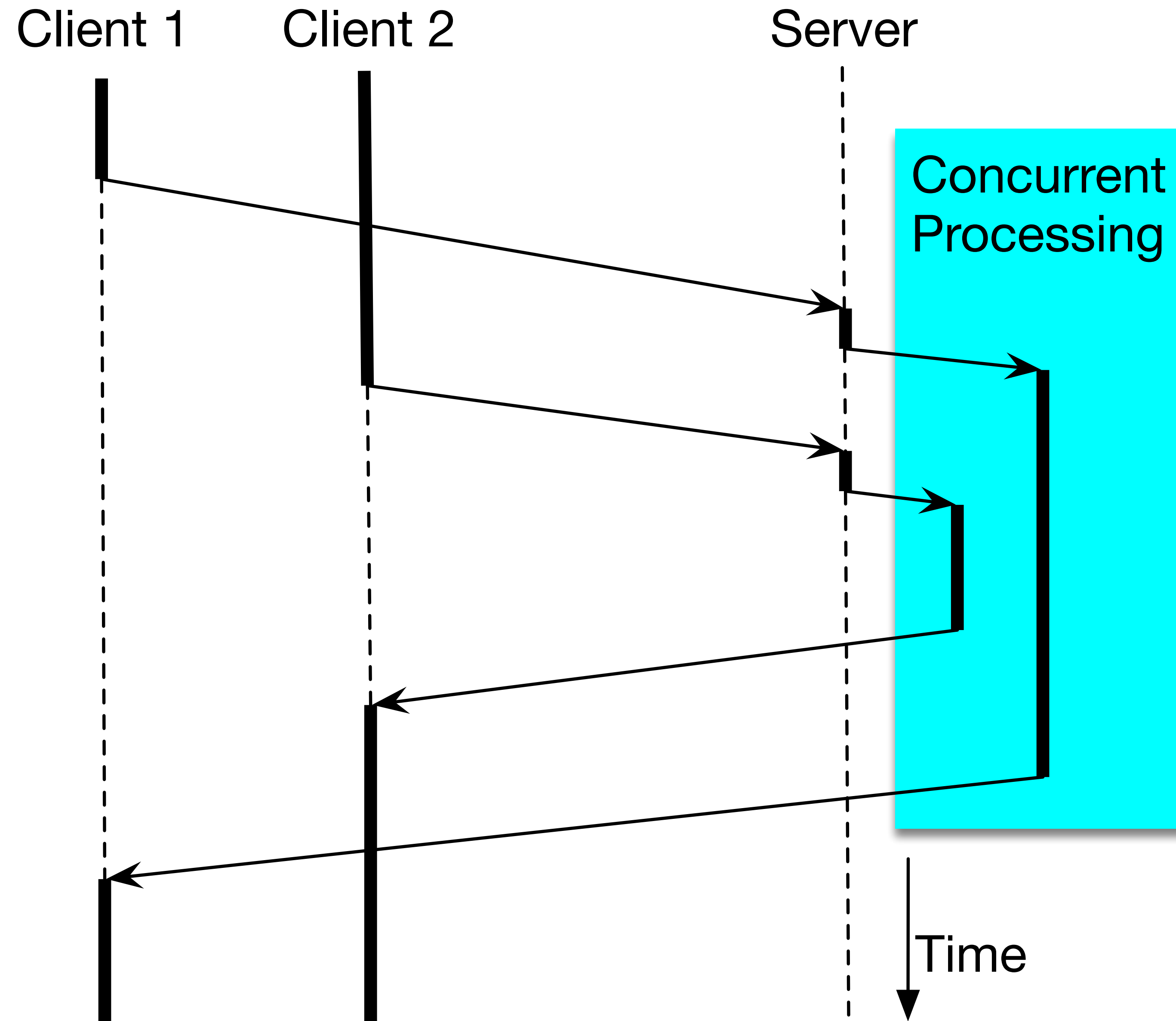
- For simple applications, GraphQL is **overkill**
- The **learning curve** for GraphQL can be steep
 - Must learn concepts like schemas, resolvers, and queries/mutations
- GraphQL queries can request **any combination of fields**
 - Managing **authorization** at the field level **is complex**
 - Easier to introduce **vulnerabilities**

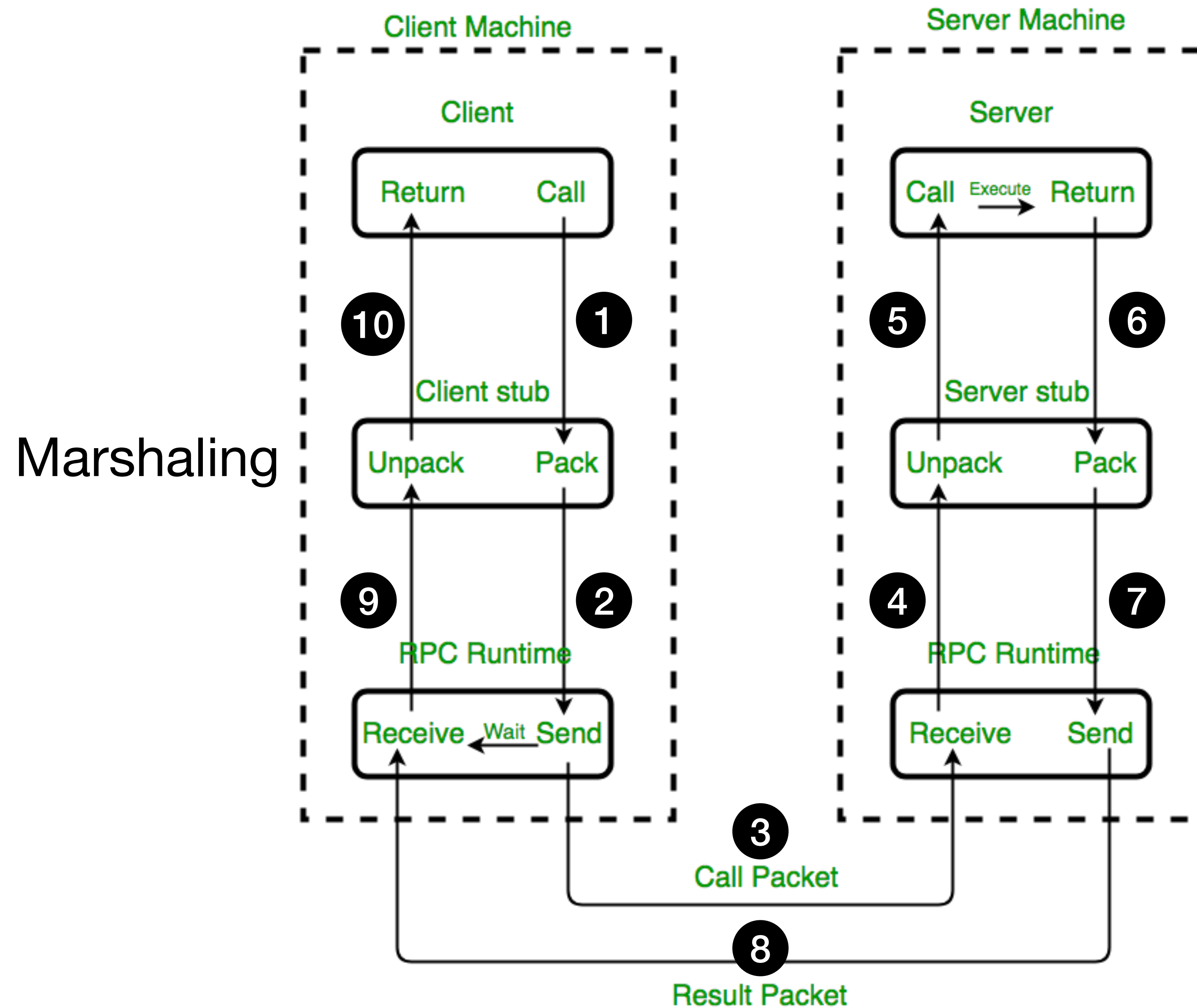
Remote Procedure Call





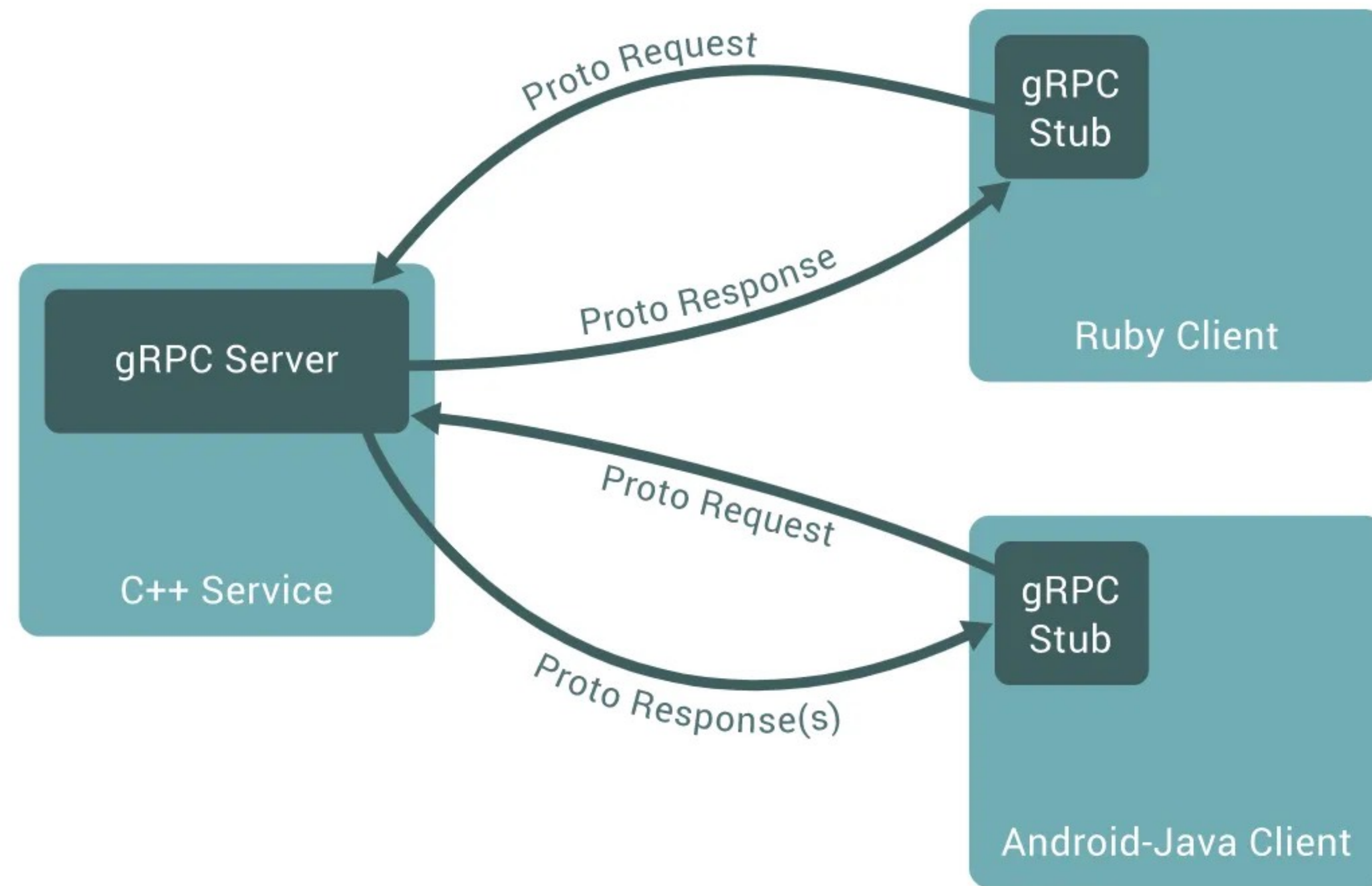
Remote Procedure Call





Google's gRPC Remote Procedure Call

gRPC is a recursive acronym



High-performance RPC framework

- Uses Protocol Buffers (protobuf) as its Interface Definition Language (IDL)
- Supports bi-directional streaming
- Uses HTTP/2 for transport, enabling multiplexed streams
- Strongly typed contracts between services with auto-generated code
- Widely used in microservices and distributed systems

Interface Definition Language (IDL)

- Specifies RPC methods and message content

```
service HelloService {
    rpc SayHello (HelloRequest) returns (HelloResponse);
}

message HelloRequest {
    string greeting = 1;
}

message HelloResponse {
    string reply = 1;
}
```

Four kinds of service methods

- Unary RPCs
 - Client sends a single request to the server
 - Get single response back
 - (like a normal function call)

```
rpc SayHello(HelloRequest) returns (HelloResponse);
```

Four kinds of service methods

- Server streaming RPCs
 - Client sends a request to the server
 - Gets a stream to read a sequence of messages back from the server
 - Client reads from the stream until there are no more messages
 - (guaranteed message ordering within an individual RPC call)

```
rpc LotsOfReplies(HelloRequest) returns (stream HelloResponse);
```

Four kinds of service methods

- Client streaming RPCs
 - Client sends a sequence of messages to the server using a stream
 - Client waits for the server to read the messages and return a response
 - (guaranteed message ordering within an individual RPC call)

```
rpc LotsOfGreetings(stream HelloRequest) returns (HelloResponse);
```

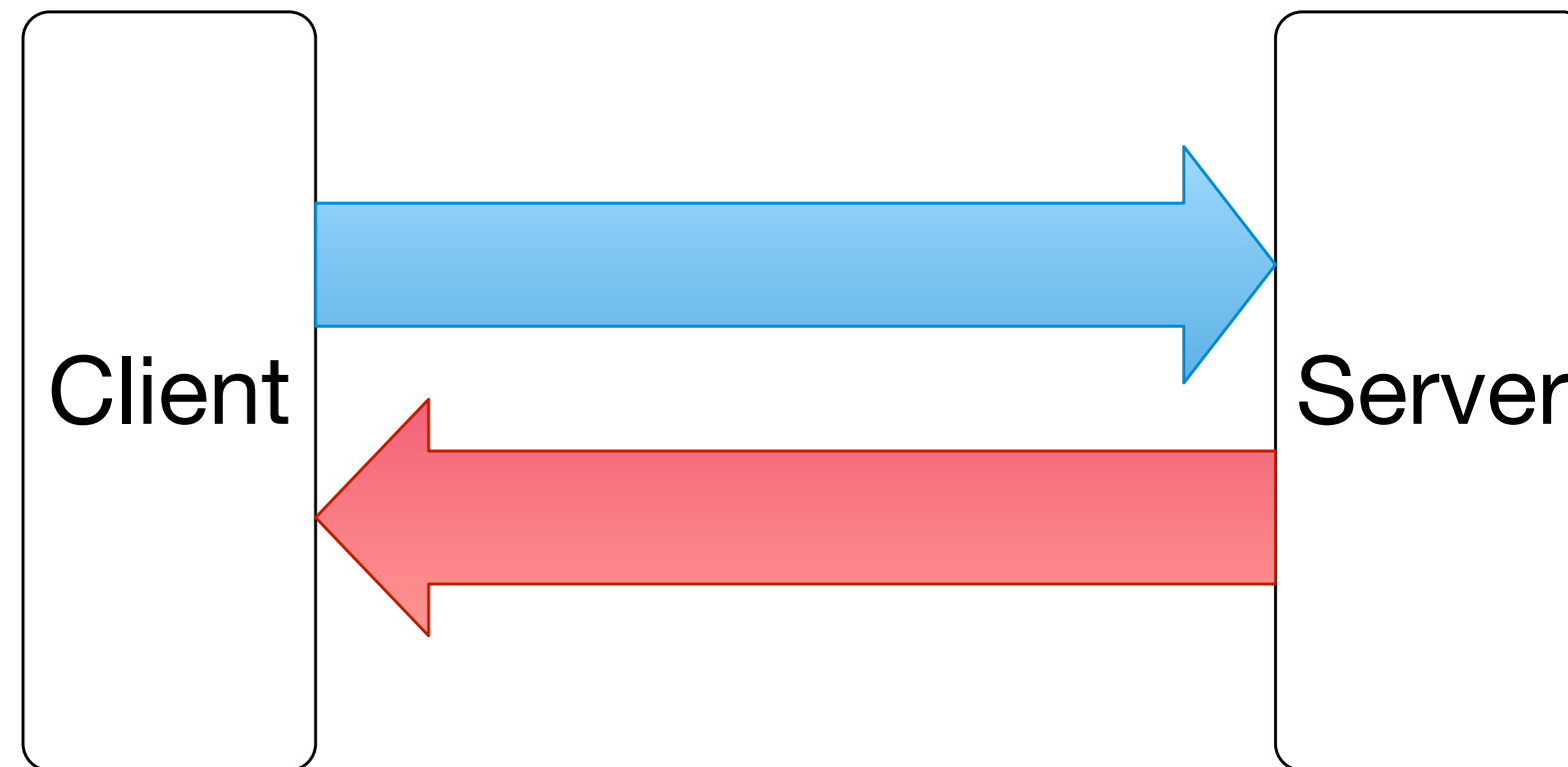
Four kinds of service methods

- Bidirectional streaming RPCs
 - Both sides send a sequence of messages using a read-write stream
 - The two streams operate independently
 - Clients and servers can read and write in whatever order they like
 - (order of messages in each stream is preserved)

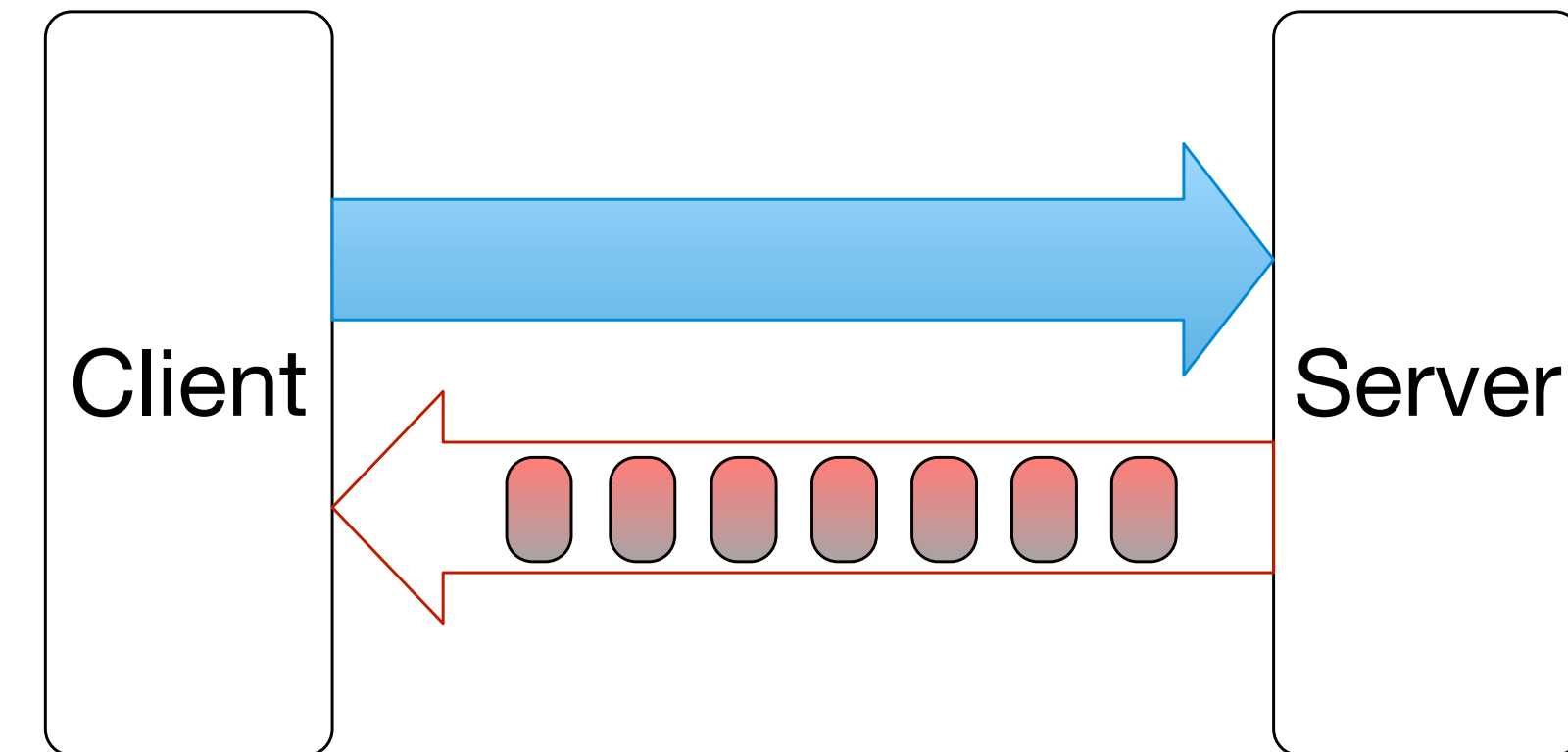
```
rpc BidiHello(stream HelloRequest) returns (stream HelloResponse);
```

gRPC Remote Procedure Calls

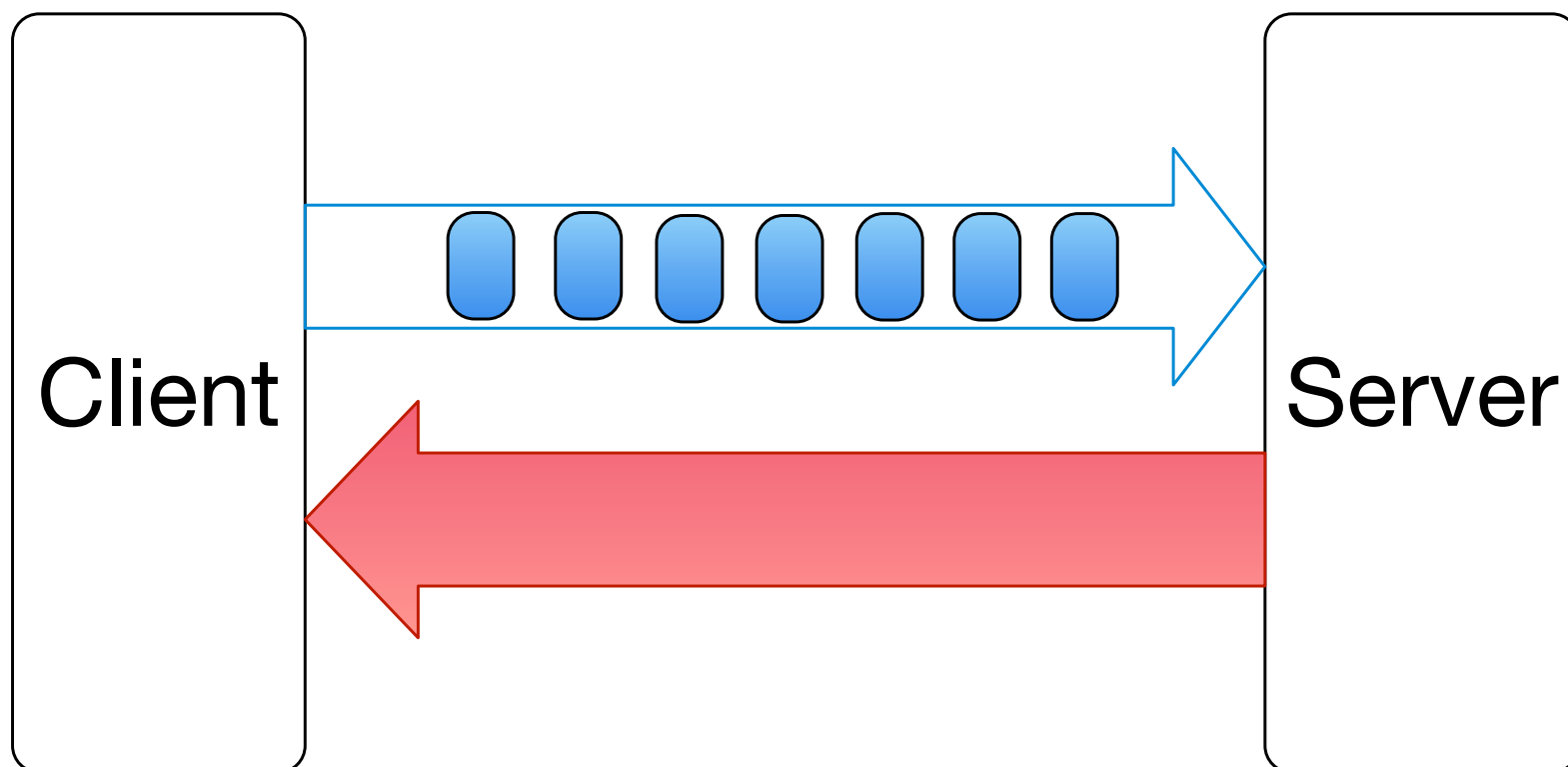
Unary



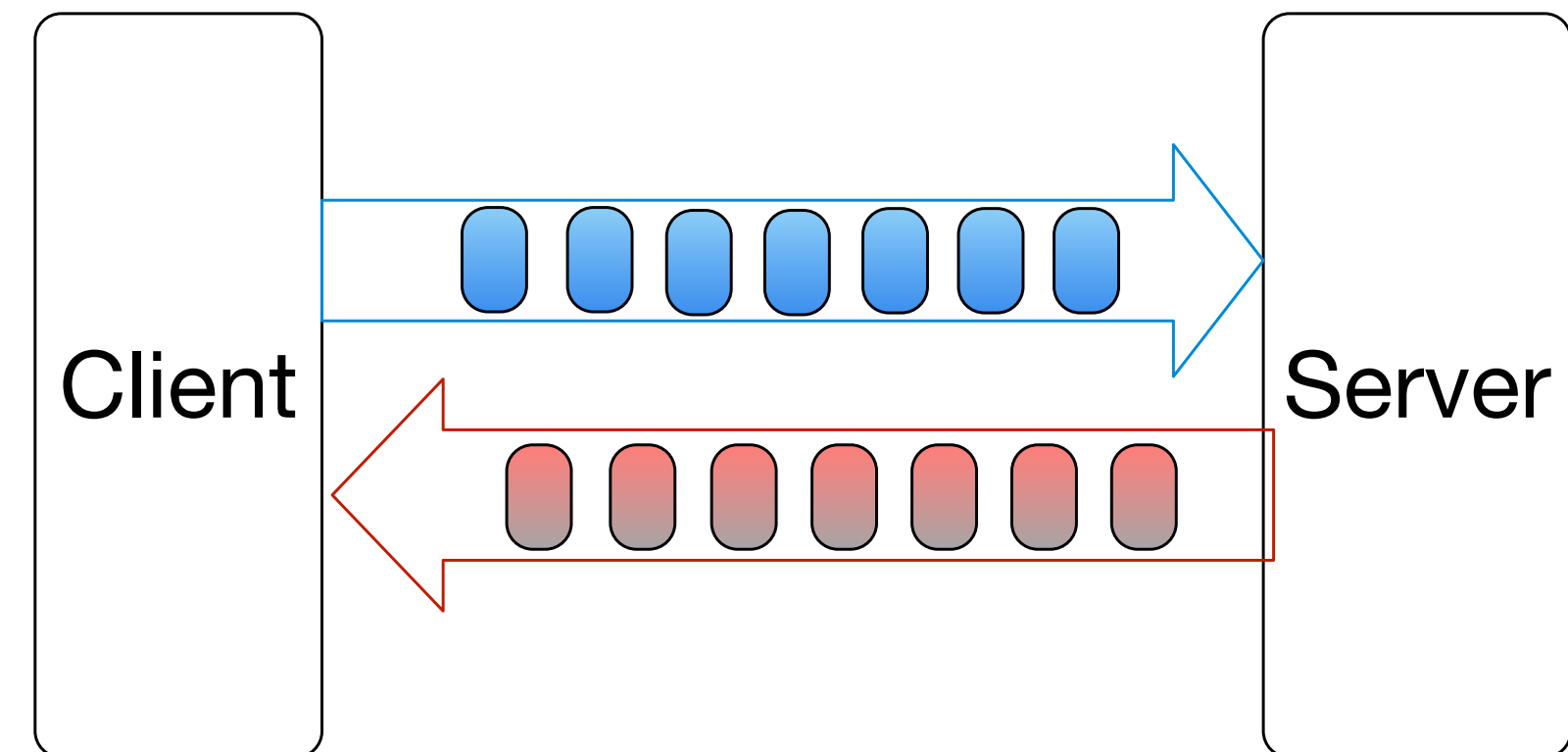
Server Streaming



Client Streaming



Bidirectional Streaming



Installing the toolchain

```
$ brew install protobuf
```

```
# Can generate code for: Java, C++, C#, Python, ...
```

```
$ protoc --version
```

```
# Go requires two additional plugins:
```

```
$ go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
```

```
$ go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```

Joke time

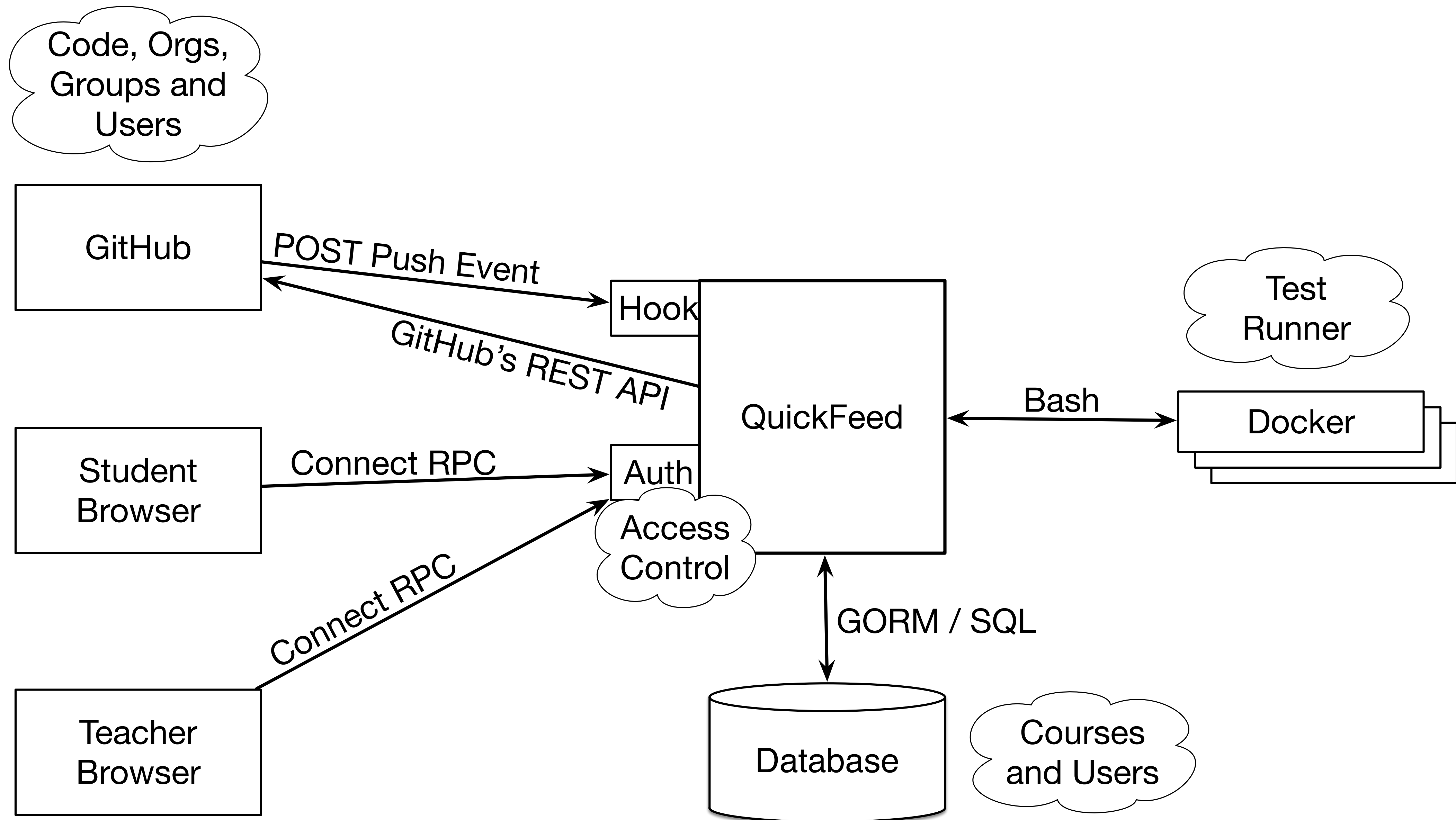
Why is gRPC so fast?

Because it skips the small
talk and gets straight to the
point with binary protobufs!

Connect RPC

Alternative to Google's gRPC implementation

- Set of libraries to build browser and **gRPC-compatible** HTTP APIs
- Generates code idiomatic and type-safe for
 - Marshaling, routing, compression, and content-type negotiation
- Supported programming languages
 - Go, JavaScript/TypeScript, Java/Kotlin, Python, Swift, and Rust



```

syntax = "proto3";
package qf;

option go_package    = "github.com/quickfeed/quickfeed/qf";
option swift_prefix = "";

import "qf/types.proto";
import "qf/requests.proto";

service QuickFeedService {
    rpc GetUser(Void) returns (User) {}
    rpc GetUsers(Void) returns (Users) {}
    rpc UpdateUser(User) returns (Void) {}

    rpc GetGroup(GroupRequest) returns (Group) {}
    rpc GetGroupsByCourse(CourseRequest) returns (Groups) {}
    rpc CreateGroup(Group) returns (Group) {}
    rpc UpdateGroup(Group) returns (Group) {}
    rpc DeleteGroup(GroupRequest) returns (Void) {}

    rpc GetCourse(CourseRequest) returns (Course) {}
    rpc GetCourses(Void) returns (Courses) {}
    rpc UpdateCourse(Course) returns (Void) {}
    rpc UpdateCourseVisibility(Enrollment) returns (Void) {}

    rpc GetAssignments(CourseRequest) returns (Assignments) {}
    rpc UpdateAssignments(CourseRequest) returns (Void) {}

    rpc GetEnrollments(EnrollmentRequest) returns (Enrollments) {}
    rpc CreateEnrollment(Enrollment) returns (Void) {}
    rpc UpdateEnrollments(Enrollments) returns (Void) {}
}
  
```

```

rpc GetSubmission(SubmissionRequest) returns (Submission) {}
rpc GetSubmissions(SubmissionRequest) returns (Submissions) {}
rpc GetSubmissionsByCourse(SubmissionRequest) returns (CourseSubmissions) {}
rpc UpdateSubmission(Grade) returns (Void) {}
rpc RebuildSubmissions(RebuildRequest) returns (Void) {}

rpc CreateReview(ReviewRequest) returns (Review) {}
rpc UpdateReview(ReviewRequest) returns (Review) {}

rpc CreateAssignmentFeedback(AssignmentFeedback) returns (Void) {}
rpc GetAssignmentFeedback(CourseRequest) returns (AssignmentFeedbacks) {}

rpc GetCourseLog(CourseLogRequest) returns (CourseLog) {}

rpc GetRepositories(CourseRequest) returns (Repositories) {}
rpc IsEmptyRepo(RepositoryRequest) returns (Void) {}
rpc SubmissionStream(Void) returns (stream Submission) {}
  
```

```

syntax = "proto3";
package qf;
option go_package = "github.com/quickfeed/quickfeed/qf";
option swift_prefix = "";

import "qf/types.proto";
import "qf/requests.proto";

service QuickFeedService {
    rpc GetUser(Void) returns (User) {}
    rpc GetUsers(Void) returns (Users) {}
    rpc UpdateUser(User) returns (Void) {}

    rpc GetGroup(GroupRequest) returns (Group) {}
    rpc GetGroupsByCourse(CourseRequest) returns (Groups) {}
    rpc CreateGroup(Group) returns (Group) {}
    rpc UpdateGroup(Group) returns (Group) {}
    rpc DeleteGroup(GroupRequest) returns (Void) {}

    rpc GetCourse(CourseRequest) returns (Course) {}
    rpc GetCourses(Void) returns (Courses) {}
    rpc UpdateCourse(Course) returns (Void) {}
    rpc UpdateCourseVisibility(Enrollment) returns (Void) {}

    rpc GetAssignments(CourseRequest) returns (Assignments) {}
    rpc UpdateAssignments(CourseRequest) returns (Void) {}

    rpc GetEnrollments(EnrollmentRequest) returns (Enrollments) {}
    rpc CreateEnrollment(Enrollment) returns (Void) {}
    rpc UpdateEnrollments(Enrollments) returns (Void) {}

```

```
// Message types that are used as requests and responses in API calls.
```

```

message CourseSubmissions {
    map<uint64, Submissions> submissions = 1;
}

```

```

message ReviewRequest {
    uint64 courseID = 1;
    Review review = 2;
}

```

```

message CourseRequest {
    uint64 courseID = 1;
}

```

```

message GroupRequest {
    uint64 courseID = 1;
    uint64 userID = 2;
    uint64 groupID = 3;
}

```

```
rpc GetCourse(CourseRequest) returns (Course) {}
rpc GetCourses(Void) returns (Courses) {}
rpc UpdateCourse(Course) returns (Void) {}
```

QuickFeed Service Implementation

```
// GetCourse returns course information for the given course.
func (s *QuickFeedService) GetCourse(ctx context.Context, in *qf.CourseRequest) (*qf.Course, error) {
    status := courseStatus(ctx, in.GetCourseID())
    course, err := s.db.GetCourseByStatus(in.GetCourseID(), status)
    if err != nil {
        qlog.FromContext(ctx).Error("failed to get course by enrollment status", label.Error, err)
        return nil, connect.NewError(connect.CodeNotFound, errors.New("course not found"))
    }
    if isTeacher(ctx, in.GetCourseID()) {
        course.Enrollments, err = s.getEnrollmentsWithActivity(in.GetCourseID())
        if err != nil {
            qlog.FromContext(ctx).Error("failed to get course enrollments with activity", label.Error, err)
            return nil, connect.NewError(connect.CodeNotFound, errors.New("failed to get course enrollments"))
        }
    }

    return course, nil
}

// GetCourses returns a list of all courses.
func (s *QuickFeedService) GetCourses(ctx context.Context, _ *qf.Void) (*qf.Courses, error) {
    courses, err := s.db.GetCourses()
    if err != nil {
        qlog.FromContext(ctx).Error("failed to get courses", label.Error, err)
        return nil, connect.NewError(connect.CodeNotFound, errors.New("no courses found"))
    }
    return &qf.Courses{
        Courses: courses,
    }, nil
}
```

```
// UpdateCourse changes the course information details.
func (s *QuickFeedService) UpdateCourse(ctx context.Context, in *qf.Course) (*qf.Void, error) {
    ctx, logger := qlog.WithLogger(ctx, label.Organization, in.GetScmOrganizationName())
    scmClient, err := s.getSCM(ctx, in.GetScmOrganizationName())
    if err != nil {
        logger.Error("failed to create SCM client", label.Error, err)
        return nil, scmConnectErr
    }
    // ensure the course exists
    _, err = s.db.GetCourse(in.GetID())
    if err != nil {
        logger.Error("failed to get course", label.Error, err)
        return nil, connect.NewError(connect.CodeNotFound, errors.New("failed to get course"))
    }
}
```

```
// ensure the organization exists
org, err := scmClient.GetOrganization(ctx, &scm.OrganizationOptions{ID: in.GetScmOrganizationID()})
if err != nil {
    logger.Error("failed to get SCM organization", label.Error, err)
    if ctxErr := logCtxErr(ctx); ctxErr != nil {
        return nil, ctxErr
    }
    if scmErr := userSCMError(err); scmErr != nil {
        return nil, scmErr
    }
    return nil, connect.NewError(connect.CodeNotFound, errors.New("failed to get organization"))
}
in.ScmOrganizationName = org.GetScmOrganizationName()

if err = s.db.UpdateCourse(in); err != nil {
    logger.Error("failed to update course", label.Error, err)
    return nil, connect.NewError(connect.CodeInvalidArgument, errors.New("failed to update course"))
}
return &qf.Void{}, nil
```

- Connect RPC libraries are committed to being 100% spec conformant with
 - gRPC and gRPC-web protocols
 - TypeScript

Get data about a specific course

```
export const getCourseData = async ({ state, effects }: Context, { courseId }: { courseId: bigint }): Promise<void> => {
  const response = await effects.global.api.client.getCourse({
    courseId,
  })
  if (response.error) {
    return
  }
  state.courseEnrollments[courseId.toString()] = response.message.enrollments
  state.groups[courseId.toString()] = response.message.groups
}
```

Get data for all courses

```
export const getCourses = async ({ state, effects }: Context): Promise<void> => {
  state.courses = []
  const response = await effects.global.api.client.getCourses({})
  if (response.error) {
    return
  }
  state.courses = response.message.courses
}
```

Updates a given course and refreshes existing courses in the state if successful

```

export const editCourse = async ({ actions, effects }: Context, { course }: { course: Course }): Promise<void> => {
  const response = await effects.global.api.client.updateCourse(course)
  if (response.error) {
    return
  }
  await actions.global.getCourses()
}
    
```

Buf is a tool for Protobuf API management

- Formatting
- Linting
- Breaking change detection
- Code generation

<https://buf.build/>

`brew install bufbuild/buf/buf`

Stores and Manages Protobuf files

- Versioned modules
- Browseable UI
- Dependency management
- API validation
- Generated documentation
- Extensible plugin system

<https://buf.build/>

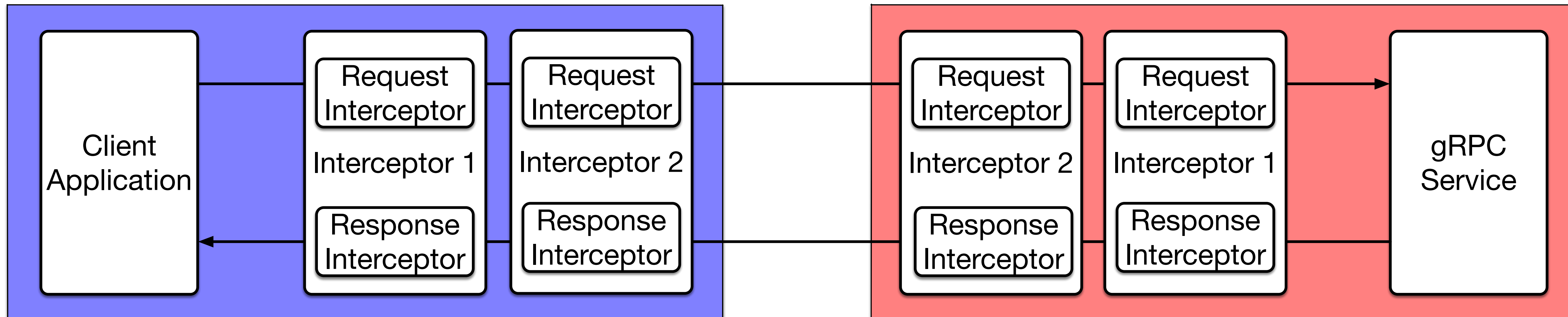
`brew install bufbuild/buf/buf`

Quiz time

What's wrong with
QuickFeed's service
definition?

It's got too many RPCs!

gRPC / Connect RPC Interceptors



```
func (s *QuickFeedService) NewQuickFeedHandler() (string, http.Handler) {
    interceptors := connect.WithInterceptors(
        interceptor.NewMetricsInterceptor(),
        interceptor.NewRPCLoggingInterceptor(s.logger),
        interceptor.NewValidationInterceptor(),
        interceptor.NewTokenAuthInterceptor(s.tm, s.db),
        interceptor.NewUserInterceptor(s.tm),
        interceptor.NewAccessControlInterceptor(s.db),
        interceptor.NewContextLoggingInterceptor(),
        interceptor.NewTokenInterceptor(s.tm),
        interceptor.NewDetachInterceptor(),
    )
    return qfconnect.NewQuickFeedServiceHandler(s, interceptors)
}
```

QuickFeed's Logging Interceptors

```
func (*ContextLoggingInterceptor) WrapUnary(next connect.UnaryFunc) connect.UnaryFunc {
    return func(ctx context.Context, request connect.AnyRequest) (connect.AnyResponse, error) {
        return next(enrichRequestLogger(ctx, request.Any()), request)
    }
}

// enrichRequestLogger adds the calling user, and the requested course when the
// caller is a trusted member of it, to the request logger. Handlers obtain the
// enriched logger with qlog.FromContext and must not repeat these attributes.
func enrichRequestLogger(ctx context.Context, request any) context.Context {
    claims, ok := auth.ClaimsFromContext(ctx)
    if !ok {
        return ctx
    }
    logger := qlog.FromContext(ctx).With(label.UserID, claims.UserID)
    courseID := getCourseID(request)
    if status, trusted := claims.Courses[courseID]; courseID > 0 && trusted && status != qf.Enrollment_NONE {
        logger = logger.With(label.CourseID, courseID)
    }
    if state, ok := ctx.Value(requestLogKey{}).(*requestLog); ok {
        state.logger = logger
    }
    return qlog.NewContext(ctx, logger)
}
```

```
time=2026-08-18T09:08:19 level=DEBUG source=web/interceptor/logging.go:47 msg="RPC completed" rpc_method=/qf.QuickFeedService/GetEnrollments user_id=232 duration=2.856806ms
time=2026-08-18T09:08:19 level=DEBUG source=web/interceptor/logging.go:47 msg="RPC completed" rpc_method=/qf.QuickFeedService/GetCourse user_id=232 course_id=21
duration=1.951491ms
time=2026-08-18T09:08:19 level=DEBUG source=web/interceptor/logging.go:47 msg="RPC completed" rpc_method=/qf.QuickFeedService/GetAssignments user_id=232 course_id=16
duration=14.03287ms
time=2026-08-18T09:08:19 level=DEBUG source=web/interceptor/logging.go:47 msg="RPC completed" rpc_method=/qf.QuickFeedService/GetAssignments user_id=232 course_id=8
duration=22.593486ms
time=2026-08-18T09:08:19 level=DEBUG source=web/interceptor/logging.go:47 msg="RPC completed" rpc_method=/qf.QuickFeedService/GetAssignments user_id=232 course_id=21
duration=32.476436ms
time=2026-08-18T09:08:19 level=DEBUG source=web/interceptor/logging.go:47 msg="RPC completed" rpc_method=/qf.QuickFeedService/GetAssignments user_id=232 course_id=5
duration=37.844501ms
time=2026-08-18T09:08:20 level=DEBUG source=web/interceptor/logging.go:47 msg="RPC completed" rpc_method=/qf.QuickFeedService/GetCourses user_id=232 duration=501.704µs
time=2026-08-18T09:08:20 level=DEBUG source=web/quickfeed_service.go:369 msg="fetching submissions" rpc_method=/qf.QuickFeedService/GetSubmissions user_id=232 course_id=5
target_user_id=232 group_id=0
time=2026-08-18T09:08:20 level=DEBUG source=web/quickfeed_service.go:369 msg="fetching submissions" rpc_method=/qf.QuickFeedService/GetSubmissions user_id=232 course_id=16
time=2026-08-18T09:08:20 level=DEBUG source=web/access_control.go:48 msg="resolved enrollment" rpc_method=/qf.QuickFeedService/GetSubmissions user_id=232 course_id=16
status=STUDENT
time=2026-08-18T09:08:20 level=DEBUG source=web/interceptor/logging.go:47 msg="RPC completed" rpc_method=/qf.QuickFeedService/GetSubmissions user_id=232 course_id=16
duration=73.427174ms
```

Joke time

Why did the DevOps
engineer bring a ladder to
the data center?

Because they wanted to
reach new “heights” in
horizontal scaling!

API

Standardization

Defines a standard, language-agnostic interface to HTTP APIs

- Allow both humans and computers to
 - Discover and understand the capabilities of the service
 - Without access to source code or documentation
- Definition can be used to generate
 - Documentation, code, testing tools etc
- <https://spec.openapis.org/oas/v3.1.0.html>

Vendor Specific APIs

- <https://developers.google.com/apis-explorer/>
- <https://developers.google.com/discovery>
- <https://developers.google.com/api-client-library>
- <https://github.com/googleapis/google-api-go-client>
- <https://github.com/googleapis/google-api-go-client/blob/main/GettingStarted.md>

- Google
 - Google Compute Engine, AutoML API, Google Calendar API, Cloud Filestore API, Cloud Firestore API,...
- Amazon APIs
 - AWS Cloud Control, EventBridge, Amazon S3
- Azure APIs
 - Azure Communication Services, Azure Cognitive Services, and Azure API Management

Amazon S3 Storage

Amazon S3

Main Uses

- Store files as objects in buckets
- Host application assets and user uploads
- Build data lakes for analytics and AI
- Back up, restore, and archive data
- Store logs, IoT data, and media
- Serve static website content



[Amazon S3 Overview \(link\)](#)

Amazon S3

Highlights

- Virtually unlimited storage capacity
- 99.999999999% data durability
- Pay only for storage and requests
- Multiple cost-optimized storage classes
- Encryption, versioning, IAM, and replication
- BBC: 25 PB historical media archive



[Amazon S3 Overview \(link\)](#)

Amazon S3 Storage (Code Example)

```
// Package main provides an example of how to use the gofakes3 library to
// create a fake S3 server for testing purposes. It demonstrates how to
// set up the server, configure the AWS SDK v2 client, create a bucket,
// and upload an object to the fake S3 server.
```

```
package main
```

```
import (
```

```
    "bytes"
```

```
    "context"
```

```
    "fmt"
```

```
    "net/http/httptest"
```

```
    "strings"
```

```
    "github.com/aws/aws-sdk-go-v2/aws"
```

```
    "github.com/aws/aws-sdk-go-v2/credentials"
```

```
    "github.com/aws/aws-sdk-go-v2/service/s3"
```

```
    "github.com/johannesboyne/gofakes3"
```

```
    "github.com/johannesboyne/gofakes3/backend/s3mem"
```

```
)
```

Amazon S3 Storage (Code Example)

```
func main() {
    backend := s3mem.New()
    server := httptest.NewServer(gofakes3.New(backend).Server())
    defer server.Close()

    cfg := aws.Config{
        Region:      "us-east-1",
        Credentials: credentials.NewStaticCredentialsProvider("test", "test", ""),
    }

    client := s3.NewFromConfig(cfg, func(o *s3.Options) {
        o.BaseEndpoint = new(server.URL)
        o.UsePathStyle = true
    })

    fmt.Printf("Fake S3 server running at %s\n", server.URL)

    ctx := context.Background()
    _, _ = client.CreateBucket(ctx, &s3.CreateBucketInput{
        Bucket: new("demo"),
    })

    fmt.Printf("Created bucket 'demo'\n")
}
```

Amazon S3 Storage (Code Example)

```
_, _ = client.PutObject(ctx, &s3.PutObjectInput{
    Bucket: new("demo"),
    Key:    new("hello.txt"),
    Body:   strings.NewReader("Hello, S3!"),
})
fmt.Printf("Uploaded object 'hello.txt' to bucket 'demo'\n")

x, _ := client.GetObject(ctx, &s3.GetObjectInput{
    Bucket: new("demo"),
    Key:    new("hello.txt"),
})
defer x.Body.Close()
buf := bytes.Buffer{}
_, _ = buf.ReadFrom(x.Body)
fmt.Printf("Downloaded object content: %s\n", buf.String())
}
```

Ceph Object Gateway is an object storage interface built on top of `librados`. It provides a RESTful gateway between applications and Ceph Storage Clusters. **Ceph Object Storage** supports two interfaces:

1. **S3-compatible:** Provides object storage functionality with an interface that is compatible with a large subset of the Amazon S3 RESTful API.
2. **Swift-compatible:** Provides object storage functionality with an interface that is compatible with a large subset of the OpenStack Swift API.

Ceph Object Storage uses the Ceph Object Gateway daemon (`radosgw`), an HTTP server designed to interact with a Ceph Storage Cluster. The Ceph Object Gateway provides interfaces that are compatible with both Amazon S3 and OpenStack Swift, and it has its own user management. Ceph Object Gateway can use a single Ceph Storage cluster to store data from Ceph File System and from Ceph Block device clients. The S3 API and the Swift API share a common namespace, which means that it is possible to write data to a Ceph Storage Cluster with one API and then retrieve that data with the other API.



Questions?

- <https://grpc.io/>
- <https://buf.build/docs/introduction>
- <https://buf.build/>
- <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>